

All 71 PYTHON Built-in Functions Explained

by Pycode.Hubb



Math & numbers

✿abs()

Returns the absolute (positive) value of a number.

```
abs(-9)          # 9
abs(3.5)         # 3.5
```

✿divmod()

Returns both quotient and remainder as a tuple.

```
divmod(10, 3)     # (3, 1) → 10÷3 = 3 rem 1
```

✿max()

Returns the largest value from an iterable or arguments.

```
max(3, 7, 2)      # 7
max([10, 5, 8])   # 10
```

✿min()

Returns the smallest value from an iterable or arguments.

```
min(3, 7, 2)      # 2
min([10, 5, 8])   # 5
```

Math & numbers

✿pow()

Raises a number to a power; optional third arg for modulo.



```
pow(2, 8)          # 256 (2^8)
pow(2, 8, 10)      # 6 (256 % 10)
```

✿round()

Rounds a number to a given number of decimal places.



```
round(3.14159, 2)  # 3.14
round(2.5)         # 2
```

✿sum()

Adds all items in an iterable, with an optional start value.



```
sum([1, 2, 3])     # 6
sum([1, 2, 3], 10) # 16 (start=10)
```

Type conversion

✿bin()

Converts an integer to its binary string representation.

```
bin(10)          # '0b1010'
bin(255)         # '0b11111111'
```

✿bool()

Converts a value to True or False.

```
bool(0)          # False
bool("hi")       # True
bool([])         # False (empty = False)
```

✿bytearray()

Creates a mutable sequence of bytes.

```
bytearray(3)      # bytearray(b'\x00\x00\x00')
bytearray(b"hi")  # bytearray(b'hi')
```

✿bytes()

Creates an immutable sequence of bytes.

```
bytes(3)          # b'\x00\x00\x00'
bytes("hi", "utf-8") # b'hi'
```

Type conversion

✿ chr()

Returns the character for a given Unicode code point.

```
chr(65)          # 'A'
chr(9829)        # '♥'
```

✿ complex()

Creates a complex number from real and imaginary parts.

```
complex(2, 3)     # (2+3j)
complex("4+5j")   # (4+5j)
```

✿ float()

Converts a string or number to a floating-point number.

```
float("3.14")     # 3.14
float(5)          # 5.0
```

✿ hex()

Converts an integer to a hexadecimal string.

```
hex(255)          # '0xff'
hex(16)           # '0x10'
```

Type conversion

✱int()

Converts a value to an integer; optional base for strings.

```
int("42")          # 42
int(3.9)           # 3 (truncates)
int("1010", 2)     # 10 (binary→int)
```

✱oct()

Converts an integer to its octal string representation.

```
oct(8)             # '0o10'
oct(64)            # '0o100'
```

✱ord()

Returns the Unicode code point of a character.

```
ord('A')           # 65
ord('❤️')           # 9829
```

✱str()

Converts any value to its string representation.

```
str(42)            # '42'
str(3.14)          # '3.14'
str(True)          # 'True'
```

Data structures

✿dict()

Creates a dictionary from keyword arguments or an iterable.

```
dict(a=1, b=2)          # {'a':1, 'b':2}
dict([("x", 10)])       # {'x':10}
```

✿frozenset()

Creates an immutable, unordered set of unique elements.

```
frozenset([1, 2, 2, 3]) # frozenset({1,2,3})
# can't add/remove items unlike set()
```

✿list()

Creates a mutable list from any iterable.

```
list("abc")             # ['a','b','c']
list(range(3))          # [0,1,2]
```

✿memoryview()

Exposes internal buffer of bytes-like objects without copying.

```
mv = memoryview(b"hello")
mv[0]                # 104 (byte value of 'h')
```


Data structures

✿object()

Returns a plain featureless base object (base of all classes).



```
o = object() # base instance
# all Python classes inherit from object()
```

✿range()

Generates a sequence of numbers (lazy, memory-efficient).



```
list(range(5))          # [0,1,2,3,4]
list(range(1, 10, 2))   # [1,3,5,7,9]
```

✿set()

Creates a mutable, unordered collection of unique elements.



```
set([1, 2, 2, 3, 3])    # {1,2,3}
set("hello")            # {'h','e','l','o'}
```

✿slice()

Creates a slice object to extract parts of sequences.



```
s = slice(1, 5)
[0, 1, 2, 3, 4, 5][s]   # [1,2,3,4]
```


Data structures

✱tuple()

Creates an immutable ordered sequence from an iterable.

```
tuple([1, 2, 3])      # (1,2,3)
tuple("abc")          # ('a','b','c')
```

*Comment 'PDF' and I'll send you
the full document in DM.*



Iteration

✿ `aiter()`

Returns an async iterator from an async iterable (Python 3.10+).



```
# used in async for loops
async for item in aiter(async_gen): ...
```

✿ `all()`

Returns True if all elements in the iterable are truthy.



```
all([True, 1, "hi"]) # True
all([True, 0, "hi"]) # False (0 is falsy)
```

✿ `anext()`

Gets the next item from an async iterator (Python 3.10+).



```
val = await anext(async_iter)
# like next() but for async iterators
```

✿ `any()`

Returns True if at least one element in the iterable is truthy.



```
any([0, False, 3]) # True (3 is truthy)
any([0, False, ""]) # False
```

Iteration

✿ enumerate()

Adds a counter index to each item in an iterable.

```
for i, v in enumerate(["a", "b"]):  
    print(i, v)           # 0 a / 1 b
```

✿ filter()

Filters items from an iterable where the function returns True.

```
list(filter(None, [0, 1, 2, 0, 3]))      # [1, 2, 3]  
list(filter(lambda x: x > 2, [1, 2, 3, 4])) # [3, 4]
```

✿ iter()

Returns an iterator object from an iterable.

```
it = iter([1, 2, 3])  
next(it)           # 1  
next(it)           # 2
```

✿ map()

Applies a function to every item of an iterable.

```
list(map(str, [1, 2, 3]))      # ['1', '2', '3']  
list(map(lambda x: x*2, [1, 2, 3])) # [2, 4, 6]
```

Iteration

✿next()

Retrieves the next item from an iterator.

```
it = iter([10, 20, 30]) next(it) # 10
next(it, -1)                # 20 (-1 is default if exhausted)
```

✿reversed()

Returns an iterator that goes through a sequence in reverse.

```
list(reversed([1, 2, 3])) # [3,2,1]
list(reversed("abc"))     # ['c','b','a']
```

✿sorted()

Returns a new sorted list from any iterable.

```
sorted([3, 1, 2])          # [1,2,3]
sorted([3, 1, 2], reverse=True) # [3,2,1]
```

✿zip()

Pairs up elements from multiple iterables into tuples.

```
list(zip([1, 2], ["a", "b"])) # [(1,'a'),(2,'b')]
# stops at shortest iterable
```

I/O & files

✱input()

Reads a line of text entered by the user from the console.



```
name = input("Enter name: ")  
# pauses and waits for keyboard input
```

✱open()

Opens a file and returns a file object for reading or writing.



```
with open("file.txt", "r")  
    as f: text = f.read() # reads whole file
```

✱print()

Prints values to the console with optional separator and end.



```
print("hello")           # hello  
print(1, 2, sep="-")     # 1-2  
print("hi", end="!")     # hi!
```

Object & attributes

✿classmethod()

Marks a method to receive the class (not instance) as first arg.

```
class C:
    @classmethod
    def create(cls):
        return cls() # cls = C
```

✿delattr()

Deletes a named attribute from an object.

```
class Dog:
    name="Rex" d = Dog() delattr(d, "name")
# removes d.name
```

✿dir()

Lists all attributes and methods of an object.

```
dir([]) # shows all list methods dir("hello")
# shows all string methods
```

✿issubclass()

Checks if a class is a subclass of another class.

```
issubclass(bool, int) # True (bool extends int)
issubclass(int, str) # False
```

Object & attributes

✿getattr()

Gets the value of a named attribute from an object.

```
class A:
    x = 10
getattr(A, "x")          # 10
getattr(A, "z", 0)       # 0 (default if missing)
```

✿hasattr()

Returns True if an object has the specified attribute.

```
class A:
    x = 1
hasattr(A, "x")          # True
hasattr(A, "z")          # False
```

✿isinstance()

Checks if an object is an instance of a class or tuple of classes.

```
isinstance(3, int)        # True
isinstance(3, (int, float)) # True
isinstance("hi", int)     # False
```


Object & attributes

✿property()

Creates a managed attribute with getter/setter/deleter.

```
class C:
    @property
    def x(self):
        return self._x # getter
```

✿setattr()

Sets the value of a named attribute on an object.

```
class A:
    pass
a = A()
setattr(a, "name", "Sam") # a.name = 'Sam'
```

✿staticmethod()

Marks a method that belongs to the class but takes no self/cls.

```
class M:
    @staticmethod
    def add(x, y):
        return x + y
M.add(2, 3) # 5
```


Object & attributes

✿ `super()`

Returns a proxy to the parent class to call its methods.

```
class Child(Parent):
    def __init__(self):
        super().__init__() # calls Parent.__init__
```

✿ `type()`

Returns the type of an object, or creates a new class dynamically.

```
type(42)           # <class 'int'>
type("hello")      # <class 'str'>
type([])           # <class 'list'>
```

✿ `vars()`

Returns the `__dict__` (attribute dictionary) of an object or scope.

```
class A:
    x=1; y=2
vars(A)           # {'x':1,'y':2,...}
vars()            # local variables dict
```

Introspection

✿ callable()

Returns True if the object appears callable (like a function).

```
callable(print)      # True
callable(42)          # False
callable(lambda:0)    # True
```

✿ globals()

Returns the current global symbol table as a dictionary.

```
x = 10
globals()["x"]        # 10
# reflects all globally defined names
```

✿ hash()

Returns the integer hash value of an immutable object.

```
hash("hello")        # some integer (fixed per run)
hash(42)              # 42
hash([1, 2])          # TypeError — lists are unhashable
```

Introspection

✿ help()

Displays the documentation/help for any object or function.



```
help(len) # shows len() docs
help(str) # shows all string methods + docs
```

✿ id()

Returns the unique memory identity (address) of an object.



```
a = "hello"
id(a) # e.g. 140234567
# two vars with same id share same object
```

✿ locals()

Returns the current local symbol table as a dictionary.



```
def f():
    x = 5
    return locals() # {'x': 5}
```

✿ repr()

Returns a developer-friendly string representation of an object.



```
repr("hello") # "'hello'" (with quotes)
repr([1, 2, 3]) # '[1, 2, 3]'
```

Code execution

✿ breakpoint()

Drops into the debugger (pdb) at the point it's called.

```
●●●
x = 10
breakpoint() # pauses here for debugging
y = x + 5
```

✿ compile()

Compiles source code string into a code object for exec/eval.

```
●●●
code = compile("x+1", "<str>", "eval")
eval(code) # evaluates the compiled code
```

✿ eval()

Evaluates a single Python expression string and returns result.

```
●●●
eval("2 + 2") # 4
eval("len('hello')") # 5
```

✿ exec()

Executes a block of Python code (statements, not just expressions).

```
●●●
exec("x = 5\nprint(x)") # prints 5
# can run multi-line code strings
```

Code execution

✱ `__import__()`

Low-level import mechanism; normally use the `import` statement instead.



```
math = __import__("math")  
math.sqrt(16) # 4.0  
# prefer: import math
```

String & display

✿ `ascii()`

Returns a string with non-ASCII characters escaped.

```
●●●
ascii("héllö")      # "'h\\xe9llo'"
ascii("hello")      # "'hello'"
```

✿ `format()`

Formats a value using a format specification string.

```
●●●
format(3.14159, ".2f") # '3.14'
format(255, "x")      # 'ff' (hex)
```

✿ `len()`

Returns the number of items in an object.

```
●●●
len("hello")          # 5
len([1, 2, 3])        # 3
len({"a":1})          # 1
```